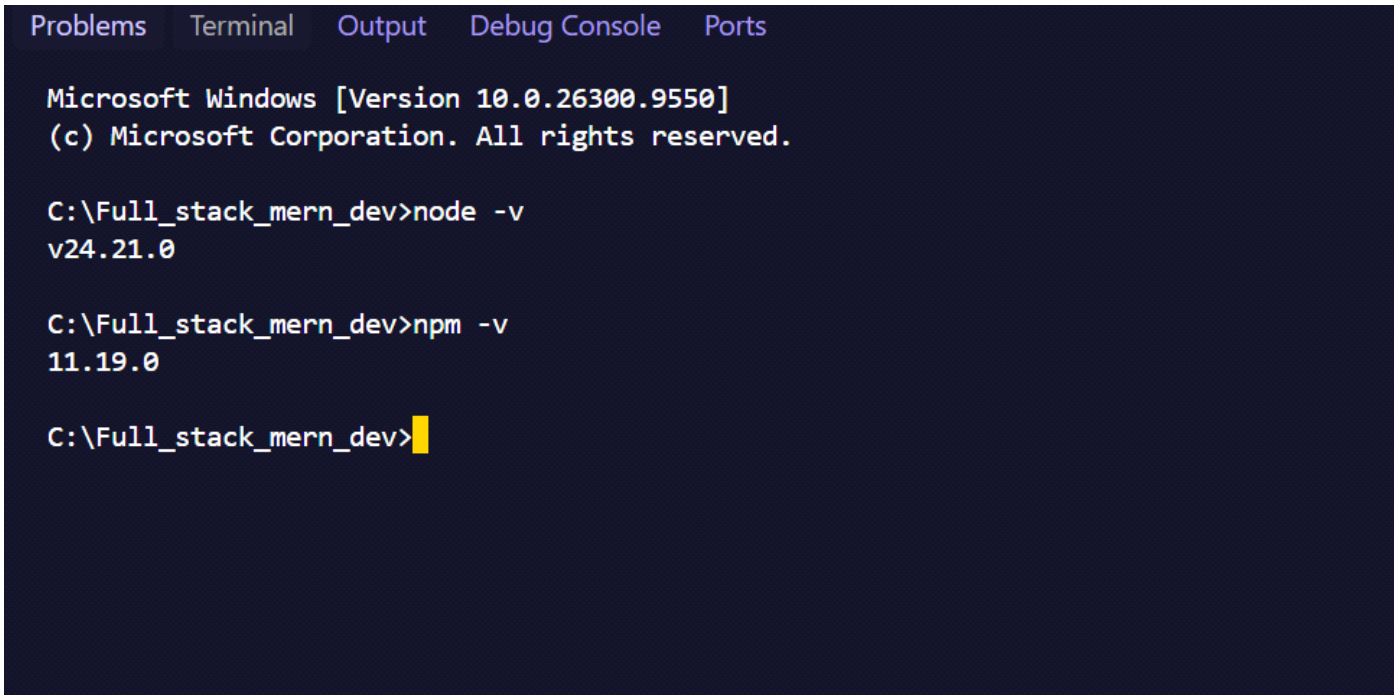
Practical No - 1.Environment Setup and JavaScript Basics

12 October 2026 18:51

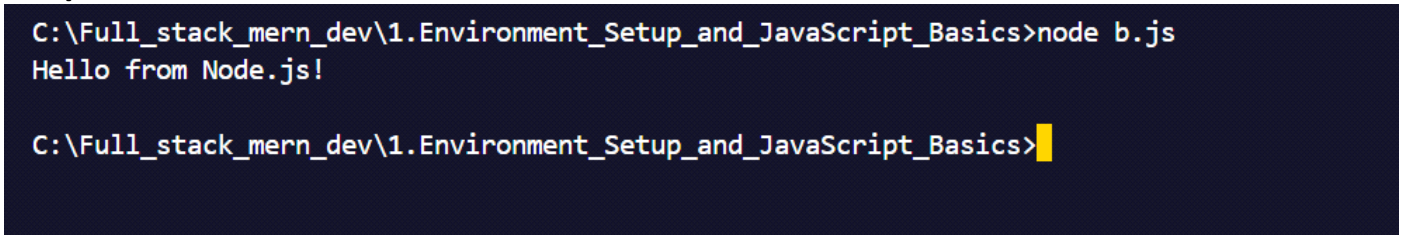
a. Install Node.js and Visual Studio Code, and verify installation.



b. Create a Simple Node.js File and Run It Using the Terminal

```
console.log("Hello from Node.js!");
```

Output -



c. Basic JavaScript Commands (Console Log, Variables, and Arithmetic)

```
// Variable declarations
const courseName = "B.Sc. IT"; // Constant (cannot be reassigned)
let studentCount = 45;         // Variable (can be updated)
// Console logging
console.log("Course:", courseName);
console.log("Initial Students:", studentCount);
// Arithmetic operations
let num1 = 15;
```

```
let num2 = 4;
let sum = num1 + num2;          // Addition (19)
let difference = num1 - num2;   // Subtraction (11)
let product = num1 * num2;      // Multiplication (60)
let quotient = num1 / num2;     // Division (3.75)
let remainder = num1 % num2;    // Modulus/Remainder (3)
console.log("--- Arithmetic Results ---");
console.log("Sum:", sum);
console.log("Difference:", difference);
console.log("Product:", product);
console.log("Quotient:", quotient);
console.log("Remainder:", remainder);
```

Output -

```
C:\Full_stack_mern_dev\1.Environment_Setup_and_JavaScript_Basics>node c._basic.js
Course: B.Sc. IT
Initial Students: 45
--- Arithmetic Results ---
Sum: 19
Difference: 11
Product: 60
Quotient: 3.75
Remainder: 3
```

d. Create and run a simple program using variables and functions.

```
// Function Declaration
function calculateRectangleArea(length, width) {
    let area = length * width;
    return area;
}
// Arrow Function (modern ES6 syntax)
const calculatePerimeter = (length, width) => {
    return 2 * (length + width);
};
// Calling the functions
let rectLength = 10;
let rectWidth = 5;
let areaResult = calculateRectangleArea(rectLength, rectWidth);
let perimeterResult = calculatePerimeter(rectLength, rectWidth);
console.log("Rectangle Dimensions:", rectLength, "x", rectWidth);
console.log("Calculated Area:", areaResult);
console.log("Calculated Perimeter:", perimeterResult);
```

Output -

```
C:\Full_stack_mern_dev\1.Environment_Setup_and_JavaScript_Basics>node d_d._Variables_and_Functions.js
Rectangle Dimensions: 10 x 5
Calculated Area: 50
Calculated Perimeter: 30

C:\Full_stack_mern_dev\1.Environment_Setup_and_JavaScript_Basics>
```

e. Demonstrate use of operators and control statements in JavaScript

```
let score = 78;
let hasSubmittedAssignment = true;
// Conditional Statements (if-else if-else)
console.log(`Student Score: \${score}`);
if (score >= 90) {
    console.log("Grade: A - Excellent!");
} else if (score >= 75) {
    console.log("Grade: B - Good job!");
} else if (score >= 50) {
    console.log("Grade: C - Needs improvement.");
} else {
    console.log("Grade: F - Fail.");
}
// Logical Operators (&& = AND, || = OR, ! = NOT)
if (score >= 50 && hasSubmittedAssignment) {
    console.log("Status: Passed the semester successfully.");
} else {
    console.log("Status: Review required.");
}
// Switch Statement
let dayOfWeek = 3;
let dayName;
switch (dayOfWeek) {
    case 1:
        dayName = "Monday";
        break;
    case 2:
        dayName = "Tuesday";
        break;
    case 3:
        dayName = "Wednesday";
        break;
    default:
        dayName = "Invalid Day";
}
console.log("Day of the week:", dayName);
```

Output -

```
C:\Full_stack_mern_dev\1.Environment_Setup_and_JavaScript_Basics>node e_Operators_and_Control_Statements.js
Student Score: ${score}
Grade: B - Good job!
Status: Passed the semester successfully.
Day of the week: Wednesday

C:\Full_stack_mern_dev\1.Environment_Setup_and_JavaScript_Basics>
```

f. Write programs using loops and template literals.

```
// 1. Template Literals Demo
let firstName = "Chinmay";
let subject = "Mearnstack_development";
let semester = 5;
// Using backticks and \${} placeholders
console.log(`--- Student Profile ---`);
console.log(`Name: ${firstName}`);
console.log(`Enrolled Subject: ${subject}`);
console.log(`Current Semester: ${semester}`);
// 2. For Loop Demo
console.log("\n--- For Loop (Counting 1 to 5) ---");
for (let i = 1; i <= 5; i++) {
    console.log(`Count index: ${i}`);
}
// 3. While Loop Demo
console.log("\n--- While Loop (Countdown) ---");
let countdown = 3;
while (countdown > 0) {
    console.log(`T-minus: ${countdown}`);
    countdown--;
}
console.log("Blast off!");
```

Output -

```
C:\Full_stack_mern_dev\1.Environment_Setup_and_JavaScript_Basics>node f_Loops_and_Template_Literals.js
--- Student Profile ---
Name: Chinmay
Enrolled Subject: Mearnstack_development
Current Semester: 5

--- For Loop (Counting 1 to 5) ---
Count index: 1
Count index: 2
Count index: 3
Count index: 4
Count index: 5

--- While Loop (Countdown) ---
T-minus: 3
T-minus: 2
T-minus: 1
Blast off!
```

Practical No - 2. Core JavaScript Concepts for MERN

12 October 2026 19:28

a. Create functions and call them with parameters.

```
// Function definition with two parameters
function calculateTotal(price, taxRate) {
    let taxAmount = price * taxRate;
    let total = price + taxAmount;
    return total;
}
// Calling the function with different arguments
let item1Total = calculateTotal(500, 0.18); // 18% tax
let item2Total = calculateTotal(1200, 0.05); // 5% tax
console.log(`Item 1 Total (with tax): ₹${item1Total}`);
console.log(`Item 2 Total (with tax): ₹${item2Total}`);
```

Output -

```
C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>node a.func_call.js
Item 1 Total (with tax): ₹590
Item 2 Total (with tax): ₹1260

C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>
```

b. Work with arrays and objects in JavaScript.

```
// 1. Working with Arrays
let techStack = ["React", "Node.js", "Express"];
techStack.push("MongoDB"); // Adds an item to the end
techStack.unshift("JavaScript"); // Adds an item to the
beginning
console.log("Full Tech Stack:", techStack);
console.log("First technology:", techStack[0]);
console.log("Total technologies:", techStack.length);
// 2. Working with Objects
let developer = {
    name: "Chinmay Palshetkar",
    role: "Full Stack Developer",
    semester: 6,
    isAvailableForHire: true
};
// Accessing object properties
console.log("\nDeveloper Profile:");
console.log(`Name: ${developer.name}`);
console.log(`Role: ${developer["role"]}`); // Alternative
bracket notation
```

Output -

```
C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>node b.arr_obj.js
Full Tech Stack: [ 'JavaScript', 'React', 'Node.js', 'Express', 'MongoDB' ]
First technology: JavaScript
Total technologies: 5

Developer Profile:
Name: Chinmay Palshetkar
Role: Full Stack Developer

C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>
```

c. Demonstrate Use of Arrow Functions

```
// Traditional function vs. Arrow function
// 1. Traditional function
function squareTraditional(num) {
    return num * num;
}
// 2. Arrow function with a single parameter and implicit
return
const squareArrow = num => num * num;
// 3. Arrow function with multiple parameters and a block
body
const calculateSalary = (basePay, bonus) => {
    let total = basePay + bonus;
    return `Total compensation: ₹${total}`;
};
console.log("Square (Traditional):",
squareTraditional(4));
console.log("Square (Arrow):", squareArrow(5));
console.log(calculateSalary(45000, 5000));
```

Output -

```
C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>node c.arro.js
Square (Traditional): 16
Square (Arrow): 25
Total compensation: ₹50000

C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>
```

d.Write a Program Using Array Methods

```
let numbers = [1, 2, 3, 4, 5];
let products = [
    { id: 1, name: "Laptop", price: 65000, inStock:
true },
    { id: 2, name: "Mouse", price: 800, inStock: false },
    { id: 3, name: "Keyboard", price: 2500, inStock:
true }
];
// 1. MAP: Transform each element (e.g., square every
number)
let squaredNumbers = numbers.map(num => num * num);
console.log("Squared Numbers:", squaredNumbers);
// 2. FILTER: Keep elements that match a condition (e.g.,
only in-stock products)
let availableProducts = products.filter(product =>
product.inStock);
console.log("Available Products:", availableProducts);
// 3. REDUCE: Accumulate values into a single result
```

```
(e.g., total price of all items)
let totalPrice = products.reduce((accumulator,
currentItem) => accumulator + currentItem.price, 0);
console.log(`Total Inventory Value: ₹${totalPrice}`);
```

Output -

```
C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>node d.arrays_methods.js
Squared Numbers: [ 1, 4, 9, 16, 25 ]
Available Products: [
  { id: 1, name: 'Laptop', price: 65000, inStock: true },
  { id: 3, name: 'Keyboard', price: 2500, inStock: true }
]
Total Inventory Value: ₹68300

C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>
```

e.Create a Program That Manipulates Objects and Displays Output

```
let userProfile = {
    username: "dev_Chinmay",
    email: "chinmay@example.com",
    theme: "dark"
};
// 1. Modifying and adding properties
userProfile.theme = "light"; // Update existing property
userProfile.lastLogin = "2026-10-12"; // Add new property
// 2. Deleting a property
delete userProfile.email;
// 3. Object Destructuring
let { username, theme } = userProfile;
console.log("Updated User Profile Object:", userProfile);
console.log(`Destructured Output -> Username: ${username},
Theme: ${theme}`);
console.log("Object Keys:", Object.keys(userProfile));
console.log("Object Values:", Object.values(userProfile));
```

Output -

```
C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>node e.mainupulate_obj_display_op.js
Updated User Profile Object: { username: 'dev_Chinmay', theme: 'light', lastLogin: '2026-10-12' }
Destructured Output -> Username: dev_Chinmay, Theme: light
Object Keys: [ 'username', 'theme', 'lastLogin' ]
Object Values: [ 'dev_Chinmay', 'light', '2026-10-12' ]

C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>
```

f. Program Combining Functions, Arrays, and Objects

```
// Sample database of products (Array of Objects)
const catalog = [
    { id: 101, name: "Mechanical Keyboard", price: 3500 },
    { id: 102, name: "Wireless Mouse", price: 1200 },
    { id: 103, name: "Type-C Hub", price: 1800 }
];
// Cart array to store selected items
let shoppingCart = [];
// Function to add a product to the cart by ID
const addToCart = (productId) => {
    const product = catalog.find(item => item.id ===
```

```
productId);
    if (product) {
        shoppingCart.push(product);
        console.log(`Added "${product.name}" to cart.`);
    } else {
        console.log("Product not found in catalog.");
    }
};
// Function to calculate cart summary using reduce
const getCartSummary = (cart) => {
    let totalItems = cart.length;
    let totalPrice = cart.reduce((sum, item) => sum +
item.price, 0);

    return {
        totalItems,
        totalPrice
    };
};
// Simulate user actions
addToCart(101);
addToCart(103);
// Display final output
let summary = getCartSummary(shoppingCart);
console.log("\n--- Shopping Cart Summary ---");
console.log(`Items in Cart: ${summary.totalItems}`);
console.log(`Total Cost: ₹${summary.totalPrice}`);
console.log("Cart Items List:", shoppingCart.map(i =>
i.name));
```

Output -

```
C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>node f.combine_func_arr_obj.js
Added "Mechanical Keyboard" to cart.
Added "Type-C Hub" to cart.

--- Shopping Cart Summary ---
Items in Cart: 2
Total Cost: ₹5300
Cart Items List: [ 'Mechanical Keyboard', 'Type-C Hub' ]

C:\Full_stack_mern_dev\2..Core_JavaScript_Concepts_for_MERN>
```

Practical No - 3.React Introduction and JSX

12 October 2026 19:48

a. Create a React application using Vite/CRA and display “Welcome to MERN Stack Development”.
b. Display dynamic data using JSX.

c. Declare variables and write a function to calculate the sum of two numbers and display the result.

d. Apply basic CSS styling to a React component

e. Create a simple functional component and render it in App.js.

f. Create a React component that displays the current date and time dynamically using JavaScript.

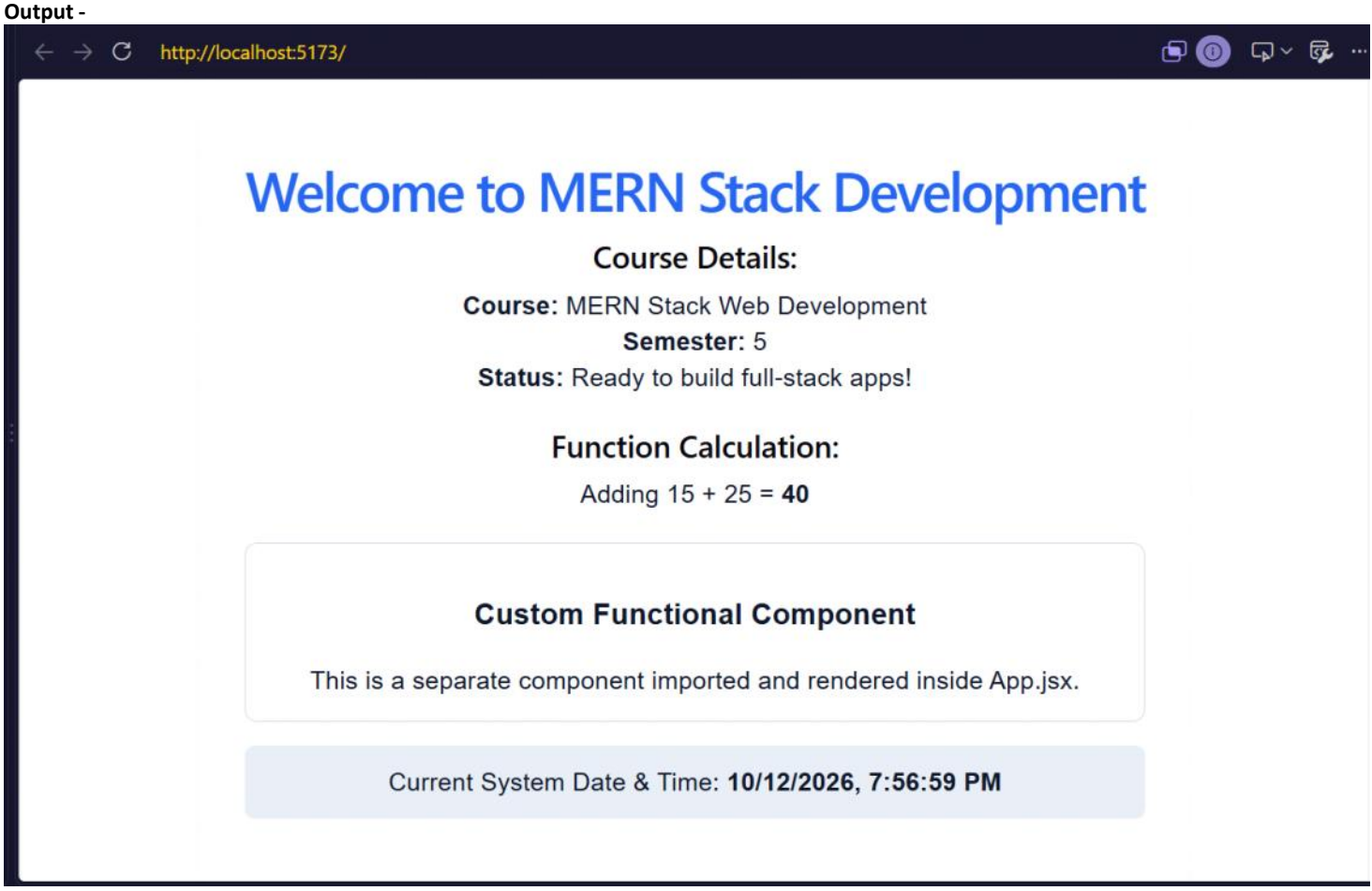
App.jsx code -

```
import React from 'react';
import './App.css';
// 1. Reusable Functional Component (Task 3e)
function GreetingCard() {
  return (
    <div style={{ background: '#f8fafc', border: '1px solid #cbd5e1', padding:
'15px', borderRadius: '8px', margin: '15px 0' }}>
      <h3>Custom Functional Component</h3>
      <p>This is a separate component imported and rendered inside
App.jsx.</p>
    </div>
  );
}
// 2. Live Date and Time Component (Task 3f)
function CurrentDateTime() {
  const currentDateTime = new Date().toLocaleString();
  return (
    <div style={{ background: '#e2e8f0', padding: '12px', borderRadius: '6px',
margin: '15px 0' }}>
      <p>Current System Date & Time: <strong>{currentDateTime}</strong></p>
    </div>
  );
}
function App() {
  // Dynamic Variables (Task 3b)
  const courseTitle = "MERN Stack Web Development";
  const semester = 5;
  const isReady = true;
  // Function to calculate sum (Task 3c)
  const num1 = 15;
  const num2 = 25;
  const calculateSum = (a, b) => a + b;
  // Inline styling object (Task 3d)
  const containerStyle = {
    maxWidth: '600px',
    margin: '40px auto',
    padding: '30px',
    fontFamily: 'sans-serif',
    borderRadius: '12px',
    boxShadow: '0 4px 6px -1px rgb(0 0 0 / 0.1)',
    background: '#ffffff',
    color: '#0f172a'
  };
  return (
    <div style={containerStyle}>
      {/* Task 3a */}
      <h1 style={{ color: '#2563eb' }}>Welcome to MERN Stack Development</h1>

      {/* Task 3b */}
      <div style={{ margin: '20px 0' }}>
        <h2>Course Details:</h2>
        <p><strong>Course:</strong> {courseTitle}</p>
        <p><strong>Semester:</strong> {semester}</p>
        <p><strong>Status:</strong> {isReady ? "Ready to build full-stack
apps!" : "Loading..."}</p>
      </div>
      {/* Task 3c */}
      <div style={{ margin: '20px 0' }}>
        <h2>Function Calculation:</h2>
        <p>Adding {num1} + {num2} = <strong>{calculateSum(num1, num2)}
```

```
</strong></p>
      </div>
      {/* Task 3e */}
      <GreetingCard />
      {/* Task 3f */}
      <CurrentDateTime />
    </div>
  );
}
export default App;
```

Output -



Practical No - 4.React Components, Props, and State

12 October 2026 20:00

a. Create multiple components and render them on a single page.
b. Pass data between components using props
c. Use the useState hook to manage and update data.
d. Create a form component to accept user input and display it dynamically.
e. Display a list of items stored in the state using map().
f. Build a small interactive component (e.g., counter or to-do list).

UserCard.jsx

```
import React from 'react';
// Receiving data via props object destructuring
function UserCard({ name, role, semester }) {
  return (
    <div style={{ border: '1px solid #cbd5e1', padding: '15px', borderRadius:
'8px', margin: '10px 0', background: '#f8fafc' }}>
      <h4>Student Profile Card</h4>
      <p><strong>Name:</strong> {name}</p>
      <p><strong>Role:</strong> {role}</p>
      <p><strong>Semester:</strong> {semester}</p>
    </div>
  );
}
export default UserCard;
```

src/Counter.jsx:

```
import React, { useState } from 'react';
function Counter() {
  // Declare state variable 'count' initialized to 0
  const [count, setCount] = useState(0);
  return (
    <div style={{ border: '1px solid #94a3b8', padding: '15px', borderRadius:
'8px', margin: '15px 0' }}>
      <h3>Interactive Counter Component</h3>
      <p>Current Count: <strong>{count}</strong></p>
      <button onClick={() => setCount(count + 1)} style={{ marginRight:
'10px', padding: '6px 12px' }}>
        Increment (+)
      </button>
      <button onClick={() => setCount(count - 1)} style={{ marginRight:
'10px', padding: '6px 12px' }}>
        Decrement (-)
      </button>
      <button onClick={() => setCount(0)} style={{ padding: '6px 12px' }}>
        Reset
      </button>
    </div>
```

```
  );
}
export default Counter;
```

UserForm.jsx:

```
import React, { useState } from 'react';
function UserForm() {
  const [inputValue, setInputValue] = useState("");
  return (
    <div style={{ border: '1px solid #94a3b8', padding: '15px', borderRadius:
'8px', margin: '15px 0' }}>
      <h3>Dynamic Form Input</h3>
      <input
        type="text"
        placeholder="Type something here..."
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
        style={{ padding: '8px', width: '80%', marginBottom: '10px' }}
      />
      <p>Live Output: <strong>{inputValue ? inputValue : "Waiting for
input..."}</strong></p>
    </div>
  );
}
export default UserForm;
```

TodoList.jsx:

```
import React, { useState } from 'react';
function TodoList() {
  const [tasks, setTasks] = useState(["Learn React Components", "Understand
Props & State", "Build MERN App"]);
  const [newTask, setNewTask] = useState("");
  const addTask = (e) => {
    e.preventDefault();
    if (newTask.trim() === "") return;
    setTasks([...tasks, newTask]); // Add new task to state array
    setNewTask(""); // Clear input
  };
  return (
    <div style={{ border: '1px solid #94a3b8', padding: '15px', borderRadius:
'8px', margin: '15px 0' }}>
      <h3>Dynamic List Rendering with map()</h3>
      <form onSubmit={addTask}>
        <input
          type="text"
          placeholder="Add new task..."
          value={newTask}
          onChange={(e) => setNewTask(e.target.value)}
          style={{ padding: '6px', marginRight: '8px' }}
        />
        <button type="submit" style={{ padding: '6px 12px' }}>Add
Task</button>
      </form>
      <ul>
        {tasks.map((task, index) => (
          <li key={index} style={{ margin: '5px 0' }}>{task}</li>
        ))}
      </ul>
```

```
    </div>
  );
}
export default TodoList;
```

src/App.jsx

```
import React from 'react';
import UserCard from './userCard';
import Counter from './counter';
import UserForm from './userForm';
import TodoList from './toolsls';
function App() {
  return (
    <div style={{ maxWidth: '650px', margin: '30px auto', padding: '20px',
fontFamily: 'sans-serif', background: '#ffffff', boxShadow: '0 4px 6px rgb(0 0
0 / 0.1)', borderRadius: '10px' }}>
      <h1 style={{ color: '#2563eb', textAlign: 'center' }}>React Components,
Props & State Lab</h1>

      {/* 4.a & 4.b */}
      <UserCard name="Chinmay" role="Full Stack AI Developer" semester={5} />
      {/* 4.c & 4.f */}
      <Counter />
      {/* 4.d */}
      <UserForm />
      {/* 4.e */}
      <TodoList />
    </div>
  );
}
export default App;
```

Output -

Student Profile Card

Name: Chinmay
Role: Full Stack AI Developer
Semester: 5

Interactive Counter Component

Current Count: **3**

Increment (+) Decrement (-) Reset

Dynamic Form Input

My Name Is Chinmay

Live Output: **My Name Is Chinmay**

Dynamic List Rend

Practical's Practical Practical s

Data Science Practicals Add Task

- Learn React Components
- Understand Props & State
- Build MERN App
- AIML Practicals

Practical No - 5.React Routing and Data Handling

13 October 2026 12:55

a. Install and configure React Router in the React application.
b. Create multiple pages and implement navigation between them.
c. Use Link and Routes to move between pages.
d. Fetch sample data from a public API using Axios.
e. Display fetched data in table or list format.
f. Implement search or filter functionality on displayed data.

App.jsx -

```
// src/App.jsx
import React from "react";
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-
dom";
import Home from "./pages/Home";
import Users from "./pages/Users";
import "./App.css";
function App() {
  return (
    <Router>
      <div style={{ maxWidth: '800px', margin: '40px auto', padding: '30px',
fontFamily: 'sans-serif', background: '#ffffff', boxShadow: '0 4px 6px rgb(0 0
0 / 0.1)', borderRadius: '12px' }}>
        <h1 style={{ color: '#2563eb', marginBottom: '10px' }}>React Routing &
API Lab</h1>

        {/* Navigation Bar */}
        <nav style={{ marginBottom: '25px', padding: '12px', background:
'#f8fafc', borderRadius: '8px', border: '1px solid #e2e8f0' }}>
          <Link to="/" style={{ marginRight: '25px', textDecoration: 'none',
color: '#2563eb', fontWeight: 'bold' }}>Home</Link>
          <Link to="/users" style={{ textDecoration: 'none', color: '#2563eb',
fontWeight: 'bold' }}>User Directory (API)</Link>
        </nav>
        {/* Routes */}
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/users" element={<Users />} />
        </Routes>
      </div>
    </Router>
  );
}
export default App;
```

Users.jsx -

```
// src/pages/Users.jsx
import React, { useState, useEffect } from 'react';
import axios from 'axios';
function Users() {
  const [users, setUsers] = useState([]);
  const [searchTerm, setSearchTerm] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null); // Added error state
  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/users')
      .then(response => {
        setUsers(response.data);
        setLoading(false);
      })
      .catch(err => {
        console.error("Error fetching data:", err);
        setError(err.message); // Capture the exact error message
        setLoading(false);
      });
  }, []);
  const filteredUsers = users.filter(user =>
    user.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
```

```
    user.email.toLowerCase().includes(searchTerm.toLowerCase())
  );
  // Display error message if the API call fails
  if (error) {
    return (
      <div style={{ padding: '20px', background: '#fee2e2', color: '#b91c1c',
borderRadius: '8px' }}>
        <h3>Failed to load user data</h3>
        <p>Error details: {error}</p>
        <p>Please check your internet connection or local proxy settings.</p>
      </div>
    );
  }
  if (loading) return <p style={{ padding: '20px' }}>Loading user directory
from public API...</p>;
  return (
    <div style={{ padding: '20px' }}>
      <h2>User Directory (API Data & Filter)</h2>

      <input
        type="text"
        placeholder="Search by name or email..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        style={{ padding: '8px', width: '100%', maxWidth: '320px',
marginBottom: '15px', borderRadius: '4px', border: '1px solid #cbd5e1' }}
      />
      <table border="1" cellPadding="12" style={{ width: '100%',
borderCollapse: 'collapse', background: '#ffffff', textAlign: 'left' }}>
        <thead>
          <tr style={{ background: '#f1f5f9', color: '#1e293b' }}>
            <th>Name</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {filteredUsers.length > 0 ? (
            filteredUsers.map(user => (
              <tr key={user.id}>
                <td>{user.name}</td>
                <td>{user.email}</td>
                <td>{user.address.city}</td>
              </tr>
            ))
          ) : (
            <tr>
              <td colSpan="3" style={{ textAlign: 'center', padding:
'20px' }}>No matching users found.</td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  );
}
export default Users;
```

Home.jsx -

```
// src/pages/Home.jsx
import React from 'react';
function Home() {
  return (
    <div>
      <h2>Home Page</h2>
      <p>Welcome to the React Router and API Integration lab assignment!</p>
    </div>
  );
}
export default Home;
```

Output -

React Routing & API Lab

Home User Directory (API)

Home Page

Welcome to the React Router and API Integration lab assignment!

React Routing & API Lab

Home User Directory (API)

User Directory (API Data & Filter)

Search by name or email...

Name	Email	City
Leanne Graham	Sincere@april.biz	Gwenborough
Ervin Howell	Shanna@melissa.tv	Wisokyburgh
Clementine Bauch	Nathan@yesenia.net	McKenziehaven
Patricia Lebsack	Julianne.OConner@kory.org	South Elvis
Chelsey Dietrich	Lucio_Hettinger@annie.ca	Roscoeview
Mrs. Dennis Schulist	Karley_Dach@jasper.info	South Christy
Kurtis Weissnat	Telly.Hoeger@billy.biz	Howemouth
Nicholas Runolfsdottir V	Sherwood@rosamond.me	Aliyaview
Glenna Reichert	Chaim_McDermott@dana.io	Bartholomebury
Clementina DuBuque	Rey.Padberg@karina.biz	Lebsackbury

Practical No - 6.Node.js and Express Fundamentals

13 October 2026 14:00

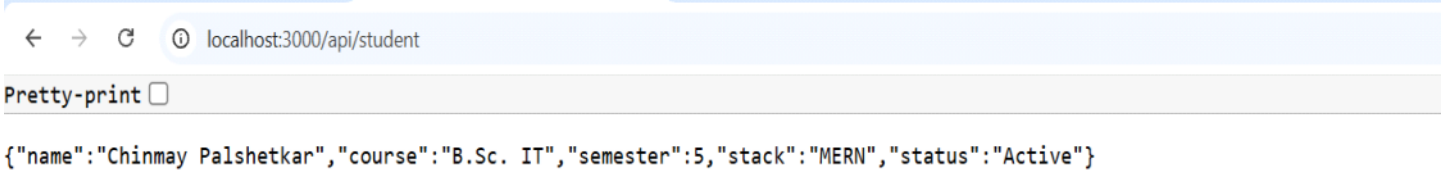
a. Install Express and create a simple server.
b. Display a message on the browser using the server response.
c. Create different routes in Express.
d. Send a JSON response from the server.
e. Test routes using browser or Postman.
f. Run the server and observe the output in the console.

Server.js -

```
// Import the express module
const express = require('express');
// Initialize the express application
const app = express();
const PORT = 3000;
// ------------------------------------------------------
// 6.b & 6.c: Defining Different Routes & Sending Text Responses
// ------------------------------------------------------
// Root route (Home)
app.get('/', (req, res) => {
    console.log("Request received for home route (/)");
    res.send('Welcome to the Express.js Backend Server!');
});
// About route
app.get('/about', (req, res) => {
    console.log("Request received for /about route");
    res.send('This is the About page served from Node.js and Express.');
});
// ------------------------------------------------------
// 6.d: Sending a JSON Response (Crucial for MERN APIs)
// ------------------------------------------------------
app.get('/api/student', (req, res) => {
    console.log("Request received for /api/student JSON route");
    res.json({
        name: "Chinmay Palshetkar",
        course: "B.Sc. IT",
        semester: 5,
        stack: "MERN",
        status: "Active"
    });
});
// ------------------------------------------------------
// 6.f: Run the Server and Observe Console Output
// ------------------------------------------------------
app.listen(PORT, () => {
    console.log(`Server is running live! Open http://localhost:${PORT} in your
browser.`);
});
```

Output -


http://localhost:3000/
Welcome to the Express.js Backend Server!


localhost:3000/api/student
Pretty-print
{"name":"Chinmay Palshetkar","course":"B.Sc. IT","semester":5,"stack":"MERN","status":"Active"}

Practical No - 7.MongoDB Basics

13 October 2026 14:17

a. Install and start MongoDB.

mongodb://localhost/): mongodb://localhost:27017 / mongosh

Output-

```
mongosh mongodb://127.0.0.

Please enter a MongoDB connection string (Default: mongodb://localhost/):

Current Mongosh Log ID: 6ace0f2a77540e190506dfff
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.13.0
Using MongoDB:          9.0.2
Using Mongosh:          2.13.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-10-13T16:08:53.294+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------
```

b. Create a database and collection.

Mongosh

use mern_lab_db

db.createCollection("students")

Output-

```
test> use mern_lab_db
switched to db mern_lab_db
mern_lab_db> db.createCollection("students")
{ ok: 1 }
```

c. Insert records in MongoDB.

```
// Insert a single student record
db.students.insertOne({
  name: "Chinmay Palshetkar",
  course: "B.Sc. IT",
  semester: 5,
  skills: ["JavaScript", "Node.js", "React", "MongoDB", "AIML", "GenAI"]
});

// Insert multiple student records
db.students.insertMany([
  { name: "Priya Verma", course: "B.Sc. IT", semester: 6, skills: ["Python", "Machine Learning"] },
  { name: "Amit Patel", course: "B.Sc. IT", semester: 5, skills: ["Java", "SQL"] },
  { name: "Priya Joshi", course: "B.Sc. IT", semester: 6, skills: ["Python", "Machine Learning"] },
  { name: "Amit Patekar", course: "B.Sc. IT", semester: 5, skills: ["Java", "SQL"] },
  { name: "Payal Verma", course: "B.Sc. IT", semester: 6, skills: ["Python", "Machine Learning"] },
  { name: "Arvind Patole", course: "B.Sc. IT", semester: 5, skills: ["Java", "SQL"] }
]);
```

Output-

```
mern_lab_db> // Insert a single student record
| db.students.insertOne({
|     name: "Chinmay Palshetkar",
|     course: "B.Sc. IT",
|     semester: 5,
|     skills: ["JavaScript", "Node.js", "React", "MongoDB", "AIML", "GenAI"]
| });
|
| // Insert multiple student records
| db.students.insertMany([
|     { name: "Priya Verma", course: "B.Sc. IT", semester: 6, skills: ["Python", "Machine Learning"] },
|     { name: "Amit Patel", course: "B.Sc. IT", semester: 5, skills: ["Java", "SQL"] },
|     { name: "Priya Joshi", course: "B.Sc. IT", semester: 6, skills: ["Python", "Machine Learning"] },
|     { name: "Amit Patekar", course: "B.Sc. IT", semester: 5, skills: ["Java", "SQL"] },
|     { name: "Payal Verma", course: "B.Sc. IT", semester: 6, skills: ["Python", "Machine Learning"] },
|     { name: "Arvind Patole", course: "B.Sc. IT", semester: 5, skills: ["Java", "SQL"] }
| ]);
|
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ace0fc477540e190506e001'),
    '1': ObjectId('6ace0fc477540e190506e002'),
    '2': ObjectId('6ace0fc477540e190506e003'),
    '3': ObjectId('6ace0fc477540e190506e004'),
    '4': ObjectId('6ace0fc477540e190506e005'),
    '5': ObjectId('6ace0fc477540e190506e006')
  }
}
```

d. View records from the collection.

db.students.find().pretty()

Output-

```
mern_lab_db> db.students.find().pretty()
[
  {
    _id: ObjectId('6ace0fc477540e190506e000'),
    name: 'Chinmay Palshetkar',
    course: 'B.Sc. IT',
    semester: 5,
    skills: [ 'JavaScript', 'Node.js', 'React', 'MongoDB', 'AIML', 'GenAI' ]
  },
  {
    _id: ObjectId('6ace0fc477540e190506e001'),
    name: 'Priya Verma',
    course: 'B.Sc. IT',
    semester: 6,
    skills: [ 'Python', 'Machine Learning' ]
  },
  {
    _id: ObjectId('6ace0fc477540e190506e002'),
    name: 'Amit Patel',
    course: 'B.Sc. IT',
    semester: 5,
    skills: [ 'Java', 'SQL' ]
  },
  {
    _id: ObjectId('6ace0fc477540e190506e003'),
    name: 'Priya Joshi',
    course: 'B.Sc. IT',
    semester: 6,
    skills: [ 'Python', 'Machine Learning' ]
  },
  {
    _id: ObjectId('6ace0fc477540e190506e004'),
    name: 'Amit Patekar',
    course: 'B.Sc. IT',
    semester: 5,
    skills: [ 'Java', 'SQL' ]
  },
  {
    _id: ObjectId('6ace0fc477540e190506e005'),
    name: 'Payal Verma',
    course: 'B.Sc. IT',
    semester: 6,
    skills: [ 'Python', 'Machine Learning' ]
  },
  {
    _id: ObjectId('6ace0fc477540e190506e006'),
    name: 'Arvind Patole',
    course: 'B.Sc. IT',
    semester: 5,
    skills: [ 'Java', 'SQL' ]
  }
]
```

```
mern_lab_db> db.students.find({ semester: 6 }).pretty()
[
  {
    _id: ObjectId('6ace0fc477540e190506e001'),
    name: 'Priya Verma',
    course: 'B.Sc. IT',
    semester: 6,
    skills: [ 'Python', 'Machine Learning' ]
  },
  {
    _id: ObjectId('6ace0fc477540e190506e003'),
    name: 'Priya Joshi',
    course: 'B.Sc. IT',
    semester: 6,
    skills: [ 'Python', 'Machine Learning' ]
  },
  {
    _id: ObjectId('6ace0fc477540e190506e005'),
    name: 'Payal Verma',
    course: 'B.Sc. IT',
    semester: 6,
    skills: [ 'Python', 'Machine Learning' ]
  }
]
mern_lab_db>
```

e. Update and delete records.

Update -

```
db.students.updateOne(
  { name: "Aarav Sharma" },
  { $set: { semester: 7 } }
);
```

Delete-

```
db.students.deleteOne({ name: "Amit Patel" });
```

Output-

```
mern_lab_db> db.students.updateOne(
...     { name: "Aarav Sharma" },
...     { $set: { semester: 7 } }
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
mern_lab_db> db.students.deleteOne({ name: "Amit Patel" });
{ acknowledged: true, deletedCount: 1 }
mern_lab_db>
```

f. Connect MongoDB with Node.js.

CMD/Bash - npm install mongoose

Server.js -

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;
// Middleware to parse JSON request bodies
app.use(express.json());
// Connect MongoDB with Node.js using Mongoose
mongoose.connect('mongodb://127.0.0.1:27017/mern_lab_db')
  .then(() => {
    console.log("Connected to MongoDB successfully!");
  })
  .catch((err) => {
    console.error("MongoDB connection error:", err);
  });
// Define a simple Mongoose Schema and Model
const studentSchema = new mongoose.Schema({
    name: String,
    course: String,
    semester: Number,
    skills: [String]
```

```
});
const Student = mongoose.model('Student', studentSchema);
// Test Route to view students from MongoDB in browser/Postman
app.get('/api/students', async (req, res) => {
    try {
        const students = await Student.find();
        res.json(students);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});
// Start server
app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
});
```

Output-




```
[
  {
    "_id": "6ace0fc477540e190506e000",
    "name": "Chinmay Palshetkar",
    "course": "B.Sc. IT",
    "semester": 5,
    "skills": [
      "JavaScript",
      "Node.js",
      "React",
      "MongoDB",
      "AIML",
      "GenAI"
    ]
  },
  {
    "_id": "6ace0fc477540e190506e001",
    "name": "Priya Verma",
    "course": "B.Sc. IT",
    "semester": 6,
    "skills": [
      "Python",
      "Machine Learning"
    ]
  },
  {
    "_id": "6ace0fc477540e190506e003",
    "name": "Priya Joshi",
    "course": "B.Sc. IT",
    "semester": 6,
    "skills": [
      "Python",
      "Machine Learning"
    ]
  },
  {
    "_id": "6ace0fc477540e190506e004",
    "name": "Amit Patekar",
    "course": "B.Sc. IT",
    "semester": 5,
    "skills": [
      "Java",
      "SQL"
```

Practical No - 8.API Creation and Integration

13 October 2026 17:04

a. Create a simple REST API using Express
b. Implement a GET API to retrieve data.
c. Fetch data from the API in a React application.
d. Send data from React to the server using a POST request.
e. Store data in MongoDB. f. Display stored data in React page.

Mern-Backend - Server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();
const PORT = 3000;
// Middleware
app.use(express.json()); // Parses incoming JSON requests
app.use(cors());         // Enables communication between React and Express
// 8.e: Connect MongoDB using Mongoose
mongoose.connect('mongodb://127.0.0.1:27017/mern_lab_db')
  .then(() => console.log("Connected to MongoDB successfully!"))
  .catch(err => console.error("MongoDB connection error:", err));
// Define a Mongoose Schema and Model
const itemSchema = new mongoose.Schema({
    title: { type: String, required: true },
    description: { type: String, required: true }
});
const Item = mongoose.model('Item', itemSchema);
// ------------------------------------------------
// 8.b: Implement a GET API to retrieve data from MongoDB
// ------------------------------------------------
app.get('/api/items', async (req, res) => {
    try {
        const items = await Item.find();
        res.json(items);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});
// ------------------------------------------------
// 8.d & 8.e: POST API to receive data from React and store in MongoDB
// ------------------------------------------------
app.post('/api/items', async (req, res) => {
    try {
        const newItem = new Item({
            title: req.body.title,
            description: req.body.description
        });
        const savedItem = await newItem.save();
        res.status(201).json(savedItem); // Return the saved record back to React
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
});
// 8.a: Start Express Server
app.listen(PORT, () => {
    console.log(`Express server running live at http://localhost:${PORT}`);
});
```

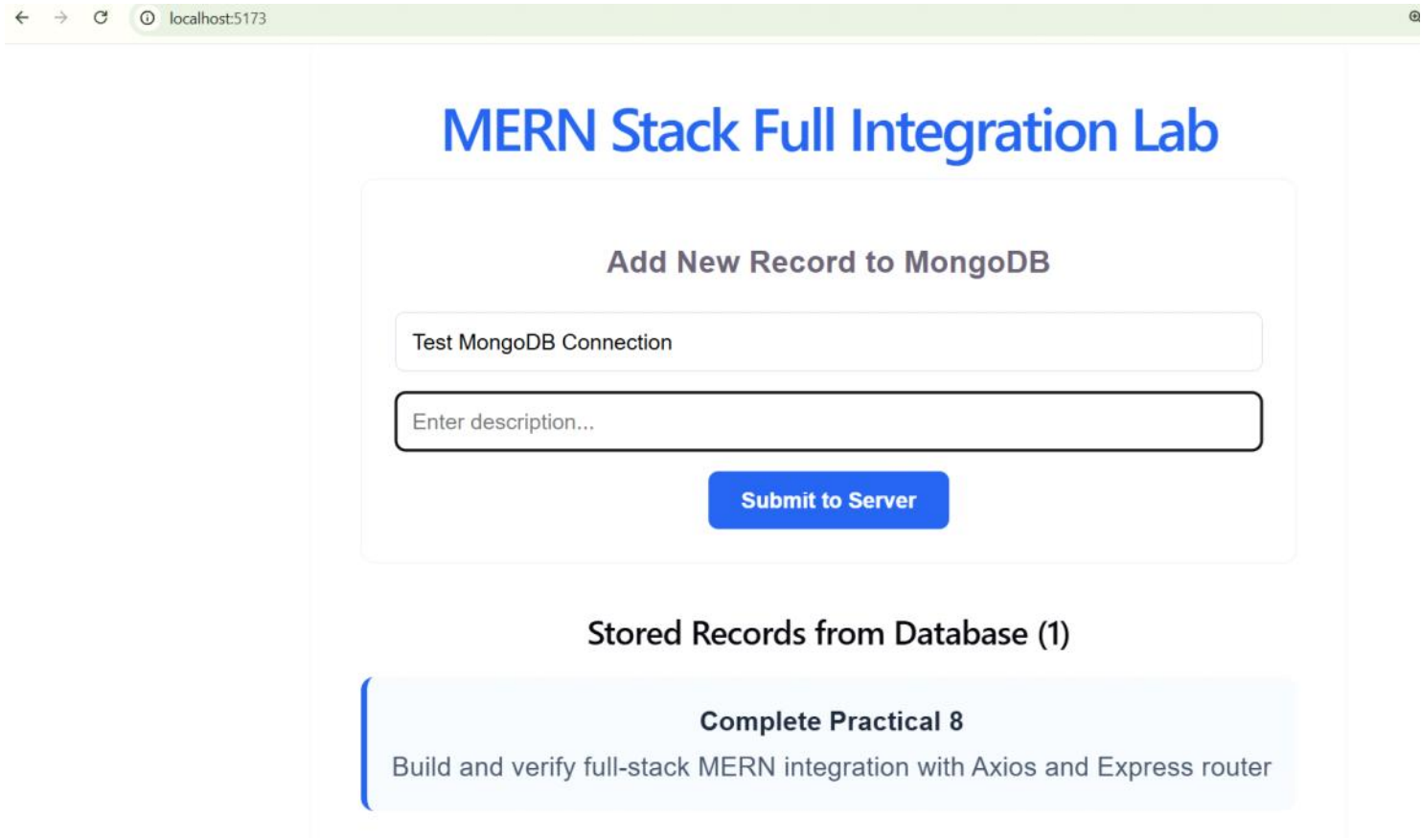
Mern-intro - Src/App.jsx -

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import './App.css';
function App() {
  const [items, setItems] = useState([]);
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  // 8.c: Fetch data from the Express REST API when the component loads
  const fetchItems = async () => {
    try {
      const response = await axios.get('http://localhost:3000/api/items');
      setItems(response.data);
    } catch (error) {
      console.error("Error fetching items:", error);
    }
  };
  useEffect(() => {
```

```
    fetchItems();
  }, []);
  // 8.d: Send data from React to the server using a POST request
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!title.trim() || !description.trim()) return;
    try {
      const response = await axios.post('http://localhost:3000/api/items', {
        title,
        description
      });
      // 8.f: Update state immediately so the new record renders on the page
      setItems([...items, response.data]);
      setTitle('');
      setDescription('');
    } catch (error) {
      console.error("Error saving item:", error);
    }
  };
  return (
    <div style={{ maxWidth: '650px', margin: '40px auto', padding: '30px', fontFamily: 'sans-serif', background: '#ffffff',
boxShadow: '0 4px 6px rgb(0 0 0 / 0.1)', borderRadius: '12px' }}>
      <h1 style={{ color: '#2563eb', textAlign: 'center' }}>MERN Stack Full Integration Lab</h1>

      {/* Form for POST request */}
      <form onSubmit={handleSubmit} style={{ marginBottom: '30px', background: '#f8fafc', padding: '20px', borderRadius:
'8px', border: '1px solid #e2e8f0' }}>
        <h3>Add New Record to MongoDB</h3>
        <div style={{ marginBottom: '12px' }}>
          <input
            type="text"
            placeholder="Enter title..."
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            style={{ width: '100%', padding: '10px', borderRadius: '6px', border: '1px solid #cbd5e1', boxSizing: 'border-
box' }}
          />
        </div>
        <div style={{ marginBottom: '12px' }}>
          <input
            type="text"
            placeholder="Enter description..."
            value={description}
            onChange={(e) => setDescription(e.target.value)}
            style={{ width: '100%', padding: '10px', borderRadius: '6px', border: '1px solid #cbd5e1', boxSizing: 'border-
box' }}
          />
        </div>
        <button type="submit" style={{ background: '#2563eb', color: '#fff', padding: '10px 20px', border: 'none',
borderRadius: '6px', fontWeight: 'bold', cursor: 'pointer' }}>
          Submit to Server
        </button>
      </form>
      {/* 8.f: Display stored data in React page */}
      <h2>Stored Records from Database ({items.length})</h2>
      {items.length === 0 ? (
        <p style={{ color: '#64748b' }}>No records found in the database yet.</p>
      ) : (
        <ul style={{ listStyleType: 'none', padding: 0 }}>
          {items.map((item) => (
            <li key={item._id} style={{ margin: '12px 0', padding: '15px', background: '#f1f5f9', borderRadius: '8px',
borderLeft: '4px solid #2563eb' }}>
              <strong style={{ fontSize: '16px', color: '#1e293b' }}>{item.title}</strong>
              <p style={{ margin: '5px 0 0 0', color: '#475569' }}>{item.description}</p>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
export default App;
```

Output -


localhost:5173
MERN Stack Full Integration Lab
Add New Record to MongoDB
Test MongoDB Connection
Enter description...
Submit to Server
Stored Records from Database (1)
Complete Practical 8
Build and verify full-stack MERN integration with Axios and Express router

Practical No - 9.User Data Management

13 October 2026 17:39

a. Create an API to add user details using Express and MongoDB.
b. Design a React form to send user data to the backend.
c. Retrieve and display user data from MongoDB.
d. Implement an update operation for existing records.
e. Implement the delete operation for selected records.
f. Display updated records dynamically in React.

Part 1: Backend Setup — Express API & MongoDB (server.js)

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();
const PORT = 3000;
// Middleware
app.use(express.json());
app.use(cors());
// Connect MongoDB (Task 7 & 9)
mongoose.connect('mongodb://127.0.0.1:27017/mern_user_db')
  .then(() => console.log("Connected to MongoDB
successfully!"))
  .catch(err => console.error("MongoDB connection error:",
err));
// Define User Schema and Model
const userSchema = new mongoose.Schema({
    name: { type: String, required: true },
    email: { type: String, required: true },
    role: { type: String, required: true }
});
const User = mongoose.model('User', userSchema);
// ------------------------------------------------
// 9.a: Create API to add user details (POST)
// ------------------------------------------------
app.post('/api/users', async (req, res) => {
    try {
        const newUser = new User(req.body);
        const savedUser = await newUser.save();
        res.status(201).json(savedUser);
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
});
// ------------------------------------------------
// 9.c: Retrieve and display user data from MongoDB (GET)
// ------------------------------------------------
app.get('/api/users', async (req, res) => {
    try {
        const users = await User.find();
        res.json(users);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});
// ------------------------------------------------
// 9.d: Implement an update operation for existing records
```

```
(PUT)
// ------------------------------------------------------
app.put('/api/users/:id', async (req, res) => {
    try {
        const updatedUser = await User.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true } // Returns the updated document
        );
        res.json(updatedUser);
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
});
// ------------------------------------------------------
// 9.e: Implement the delete operation for selected
records (DELETE)
// ------------------------------------------------------
app.delete('/api/users/:id', async (req, res) => {
    try {
        await User.findByIdAndDelete(req.params.id);
        res.json({ message: "User deleted
successfully" });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});
// Start Server
app.listen(PORT, () => {
    console.log(`Express server running live at
http://localhost:${PORT}`);
});
```

Part 2: Frontend Setup — React Form and Dynamic UI (src/App.jsx)

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import './App.css';
function App() {
  const [users, setUsers] = useState([]);
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [role, setRole] = useState('');
  const [editingId, setEditingId] = useState(null); //
Tracks if we are editing a user
  // Fetch users from API on component mount (9.c)
  const fetchUsers = async () => {
    try {
      const response = await
axios.get('http://localhost:3000/api/users');
      setUsers(response.data);
    } catch (error) {
      console.error("Error fetching users:", error);
    }
  };
  useEffect(() => {
    fetchUsers();
  }, []);
```

```
  // 9.b & 9.d: Handle Form Submission (Create new or
Update existing)
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!name.trim() || !email.trim() || !role.trim())
return;
    try {
      if (editingId) {
        // 9.d & 9.f: Update existing record and refresh
state dynamically
        const response = await
axios.put(`http://localhost:3000/api/users/${editingId}`,
{ name, email, role });
        setUsers(users.map(user => user._id ===
editingId ? response.data : user));
        setEditingId(null);
      } else {
        // 9.b & 9.f: Send new user data to backend and
add to state
        const response = await
axios.post('http://localhost:3000/api/users', { name,
email, role });
        setUsers([...users, response.data]);
      }
      // Clear input fields
      setName('');
      setEmail('');
      setRole('');
    } catch (error) {
      console.error("Error saving user:", error);
    }
  };
  // Populate form with selected user data for editing
  const handleEdit = (user) => {
    setEditingId(user._id);
    setName(user.name);
    setEmail(user.email);
    setRole(user.role);
  };
  // 9.e & 9.f: Delete selected record and update UI
dynamically
  const handleDelete = async (id) => {
    try {
      await axios.delete(`http://localhost:3000/api/users/
${id}`);
      setUsers(users.filter(user => user._id !== id));
    } catch (error) {
      console.error("Error deleting user:", error);
    }
  };
  return (
    <div style={{ maxWidth: '700px', margin: '40px auto',
padding: '30px', fontFamily: 'sans-serif', background:
'#ffffff', boxShadow: '0 4px 6px rgb(0 0 0 / 0.1)',
borderRadius: '12px' }}>
      <h1 style={{ color: '#2563eb', textAlign:
'center' }}>MERN User Management Dashboard</h1>

      {/* 9.b: React Form Component */}
      <form onSubmit={handleSubmit} style={{ marginBottom:
```

```
'30px', background: '#f8fafc', padding: '20px',
borderRadius: '8px', border: '1px solid #e2e8f0' }}>
        <h3>{editingId ? "Update User Record" : "Add New
User Details"}</h3>
        <div style={{ marginBottom: '12px' }}>
          <input
            type="text"
            placeholder="Full Name"
            value={name}
            onChange={(e) => setName(e.target.value)}
            style={{ width: '100%', padding: '10px',
borderRadius: '6px', border: '1px solid #cbd5e1',
boxSizing: 'border-box' }}
          />
        </div>
        <div style={{ marginBottom: '12px' }}>
          <input
            type="email"
            placeholder="Email Address"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            style={{ width: '100%', padding: '10px',
borderRadius: '6px', border: '1px solid #cbd5e1',
boxSizing: 'border-box' }}
          />
        </div>
        <div style={{ marginBottom: '12px' }}>
          <input
            type="text"
            placeholder="Role (e.g., Developer)"
            value={role}
            onChange={(e) => setRole(e.target.value)}
            style={{ width: '100%', padding: '10px',
borderRadius: '6px', border: '1px solid #cbd5e1',
boxSizing: 'border-box' }}
          />
        </div>
        <button type="submit" style={{ background:
editingId ? '#16a34a' : '#2563eb', color: '#fff', padding:
'10px 20px', border: 'none', borderRadius: '6px',
fontWeight: 'bold', cursor: 'pointer' }}>
          {editingId ? "Update User" : "Add User"}
        </button>
        {editingId && (
          <button
            type="button"
            onClick={() => { setEditingId(null);
setName(''); setEmail(''); setRole(''); }}
            style={{ marginLeft: '10px', background: '#
64748b', color: '#fff', padding: '10px 20px', border:
'none', borderRadius: '6px', cursor: 'pointer' }}
          >
            Cancel
          </button>
        )}
      </form>
      {/* 9.c & 9.f: Retrieve and Display Stored Records
Dynamically */}
      <h2>Registered Users Directory ({users.length})</h2>
      {users.length === 0 ? (
```

```
        <p style={{ color: '#64748b' }}>No users found in
the database.</p>
      ) : (
        <ul style={{ listStyleType: 'none', padding: 0 }}>
          {users.map((user) => (
            <li key={user._id} style={{ display: 'flex',
justifyContent: 'space-between', alignItems: 'center',
margin: '12px 0', padding: '15px', background: '#f1f5f9',
borderRadius: '8px', borderLeft: '4px solid #2563eb' }}>
              <div>
                <strong style={{ fontSize: '16px', color:
'#1e293b' }}>{user.name}</strong> <span style={{ color: '#
2563eb' }}>({user.role})</span><br />
                <span style={{ color: '#475569', fontSize:
'14px' }}>{user.email}</span>
              </div>
              <div>
                <button onClick={() => handleEdit(user)}
style={{ marginRight: '8px', background: '#ca8a04', color:
'#fff', border: 'none', padding: '6px 12px', borderRadius:
'4px', cursor: 'pointer', fontWeight: 'bold' }}>
                  Edit
                </button>
                <button onClick={() => handleDelete(user.
_id)} style={{ background: '#dc2626', color: '#fff',
border: 'none', padding: '6px 12px', borderRadius: '4px',
cursor: 'pointer', fontWeight: 'bold' }}>
                  Delete
                </button>
              </div>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
export default App;
```

Output-

MERN User Management Dashboard

Add New User Details

Full Name

Email Address

Role (e.g., Developer)

Add User

Registered Users Directory (4)

Chinmay Palshetkar (AI Developer)
chinmaypalshetkar30000@gmail.com

Edit Delete

Registered Users Directory (4)

Chinmay Palshetkar (AI Developer)
chinmaypalshetkar30000@gmail.com
Edit Delete

shabana shaikh (mern stack developer)
shbnaShaikh90@gmail.com
Edit Delete

Nasim Zain (Software developer)
NsimZ77@gmail.com
Edit Delete

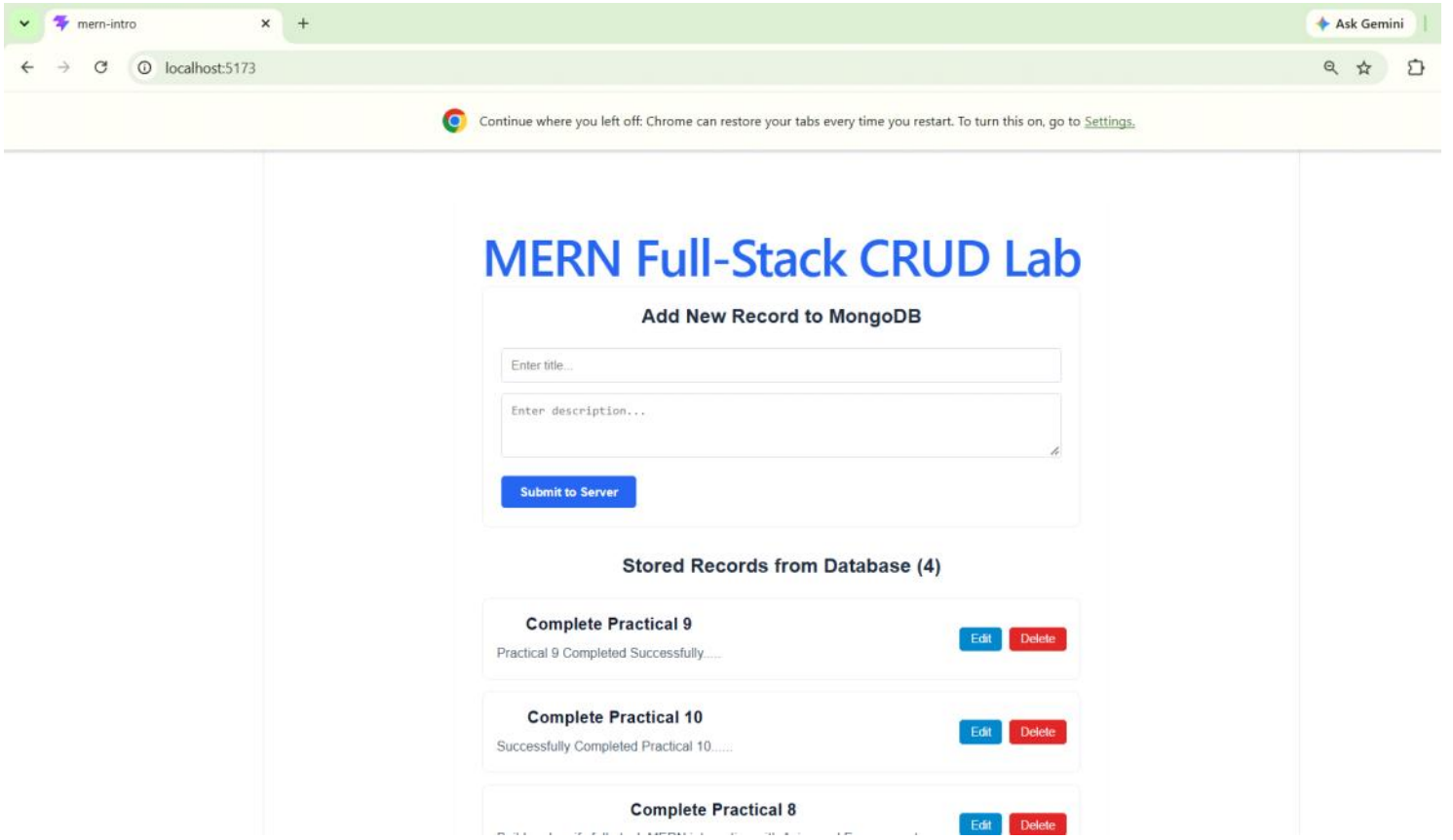
priya verma (Developer)
priyaVerma90@gmail.com
Edit Delete

Practical No - 10.Full Stack Execution and Validation

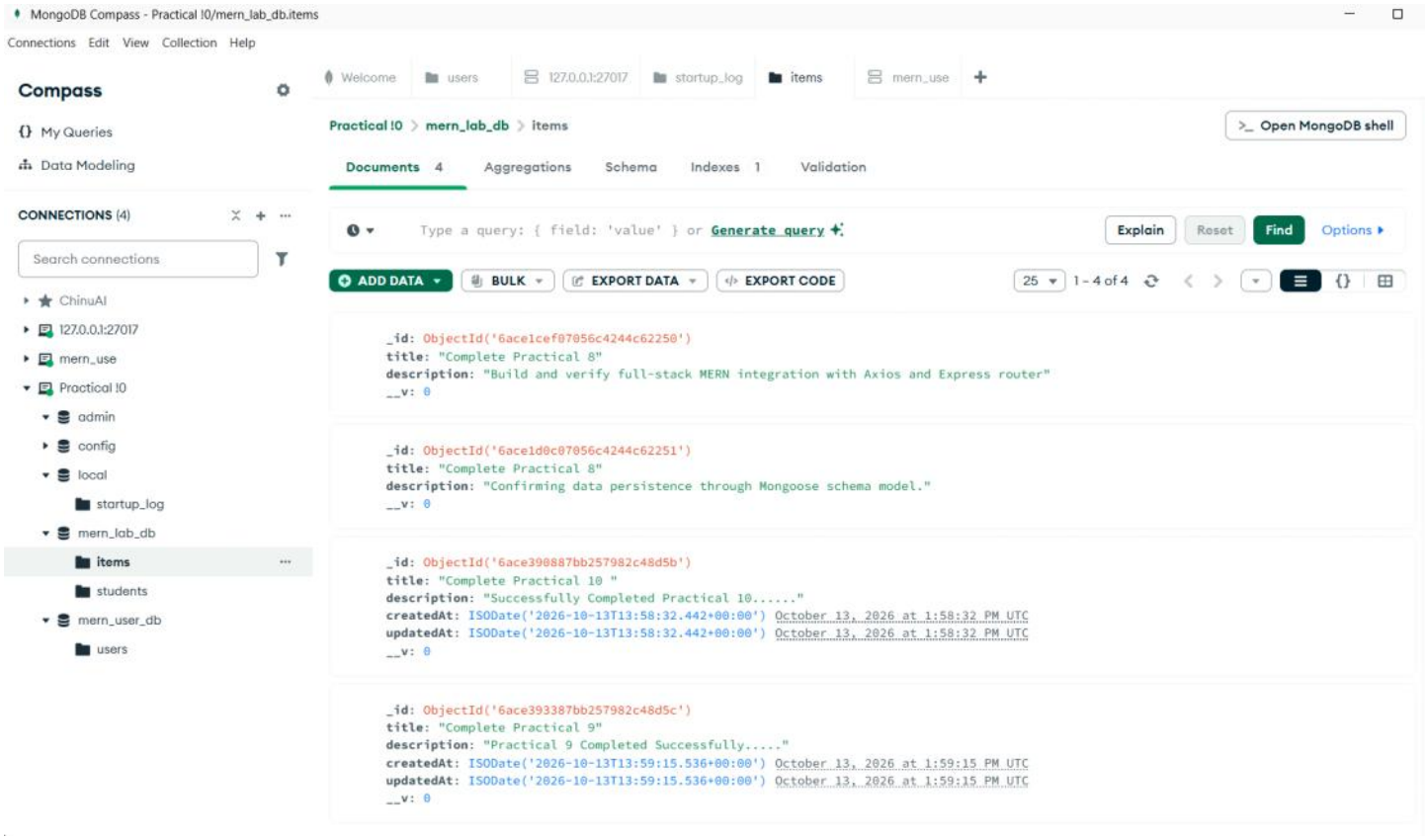
13 October 2026 17:39

a. Integrate React frontend with Node and Express backend.
b. Perform insert, update, and delete operations through UI.
c. Display database records dynamically in React.
d. Implement frontend form validation.
e. Test all APIs using Postman.
f. Execute and verify the complete working of the frontend, backend, and database together.

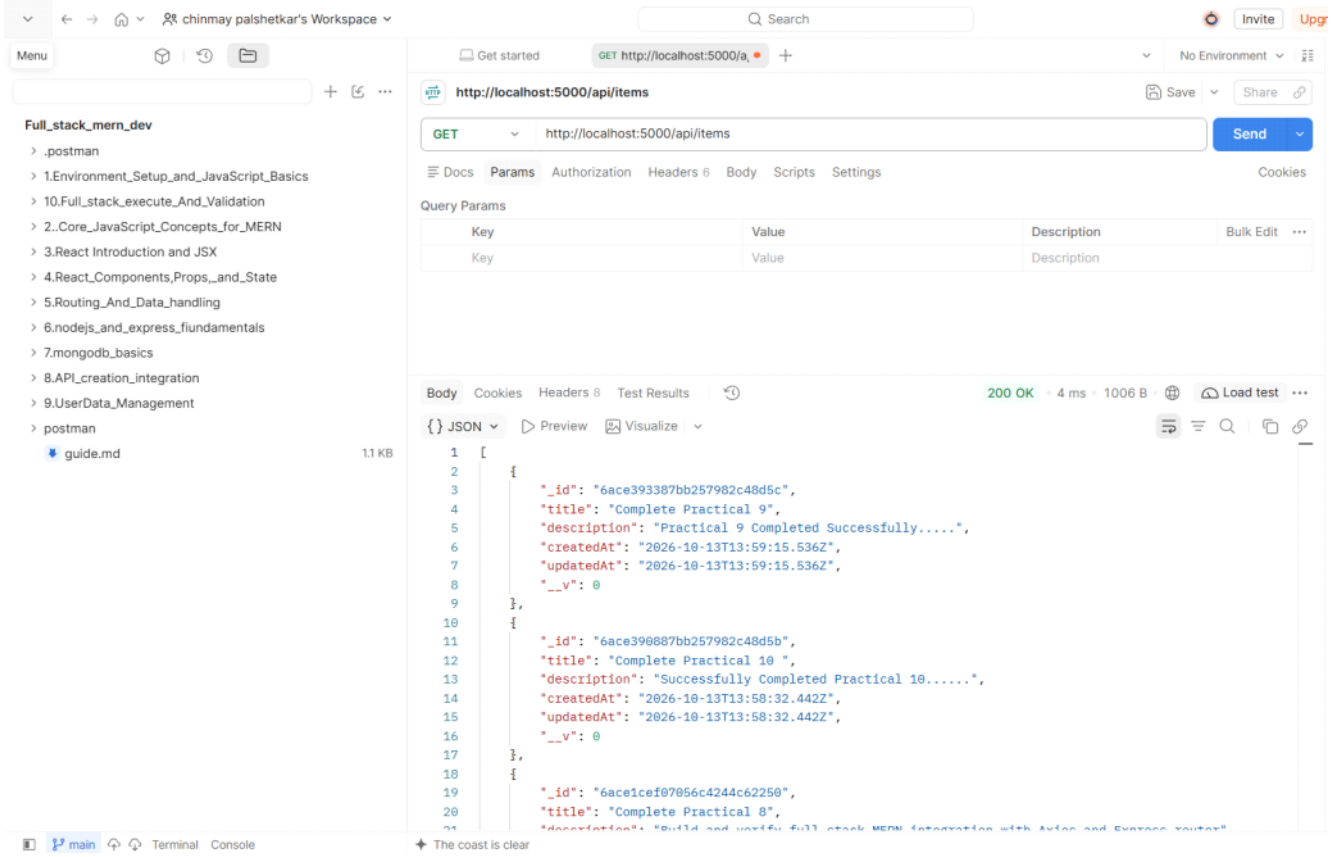
Output -
Frontend



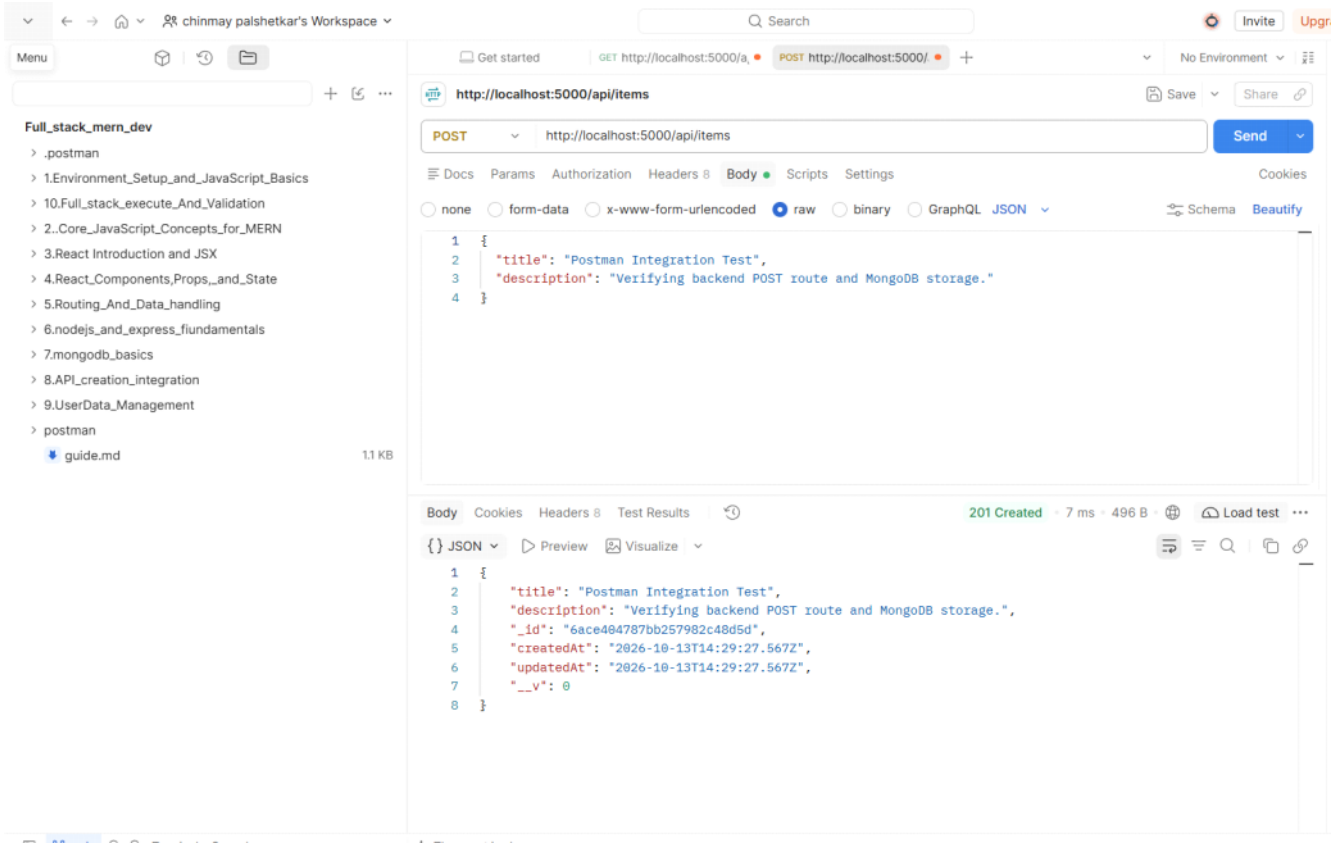
MongoDB Compass-



Postman- Get 1. Test the GET API (Fetch All Records)



2. Test the POST API (Insert a Record)



3. Test the PUT API (Update an Existing Record)

